NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #1 – class 3/5

freeRTOS: TASKS & SYNCHRONIZATION WITH SEMAPHORES

2025 - 2026

OBJECTIVES

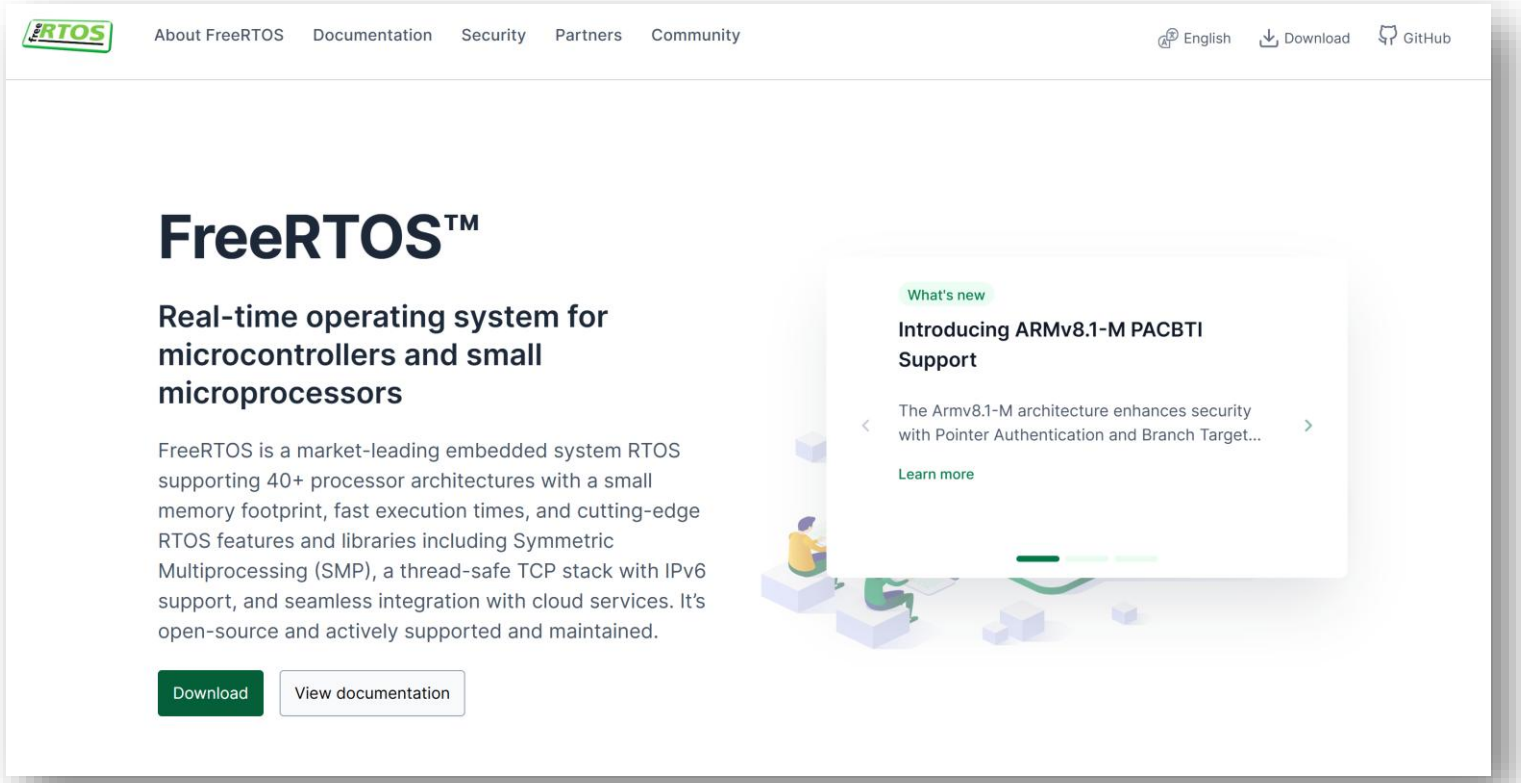
freeRTOS: Tasks and Synchronization with Semaphores

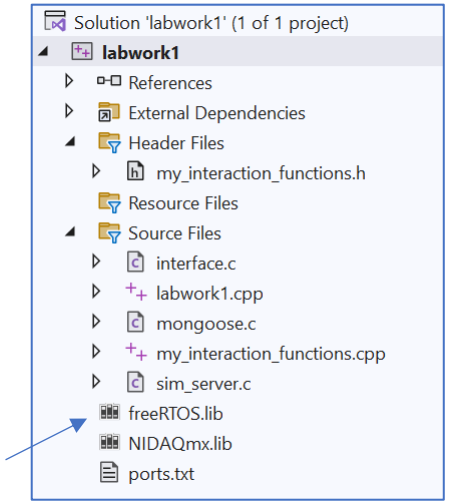
- Overview of RTOS / freeRTOS
- Task Creation
- Semaphores
- Hook functions
- Interrupts

Overview of RTOS / freeRTOS

- Real time operating system for microcontrollers and small microprocessors.

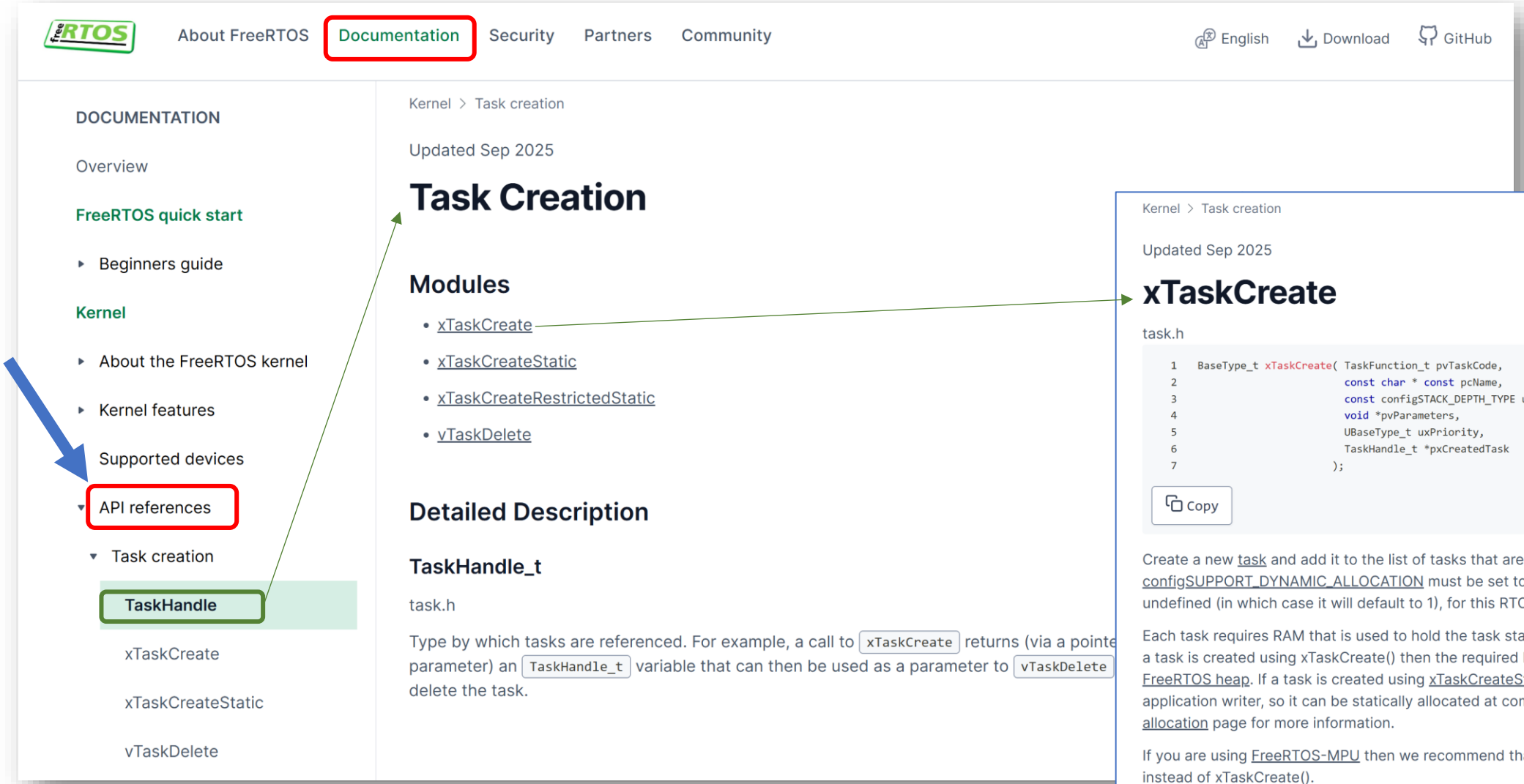
www.freertos.org





*no need to download…. You already have the "freeRTOS.lib" in your **labwork1** project*

freeRTOS API

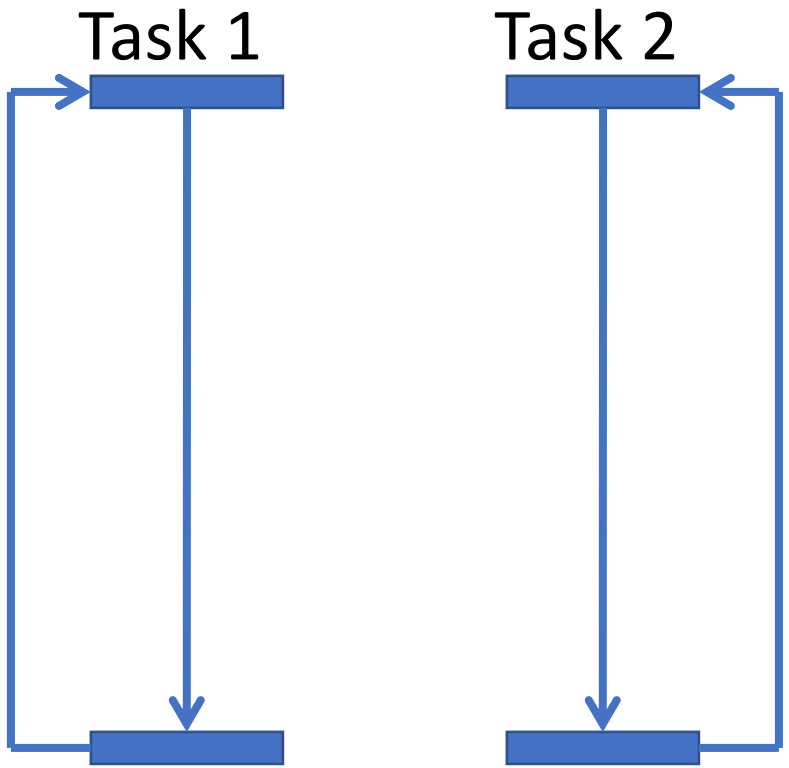


freeRTOS - Functionalities

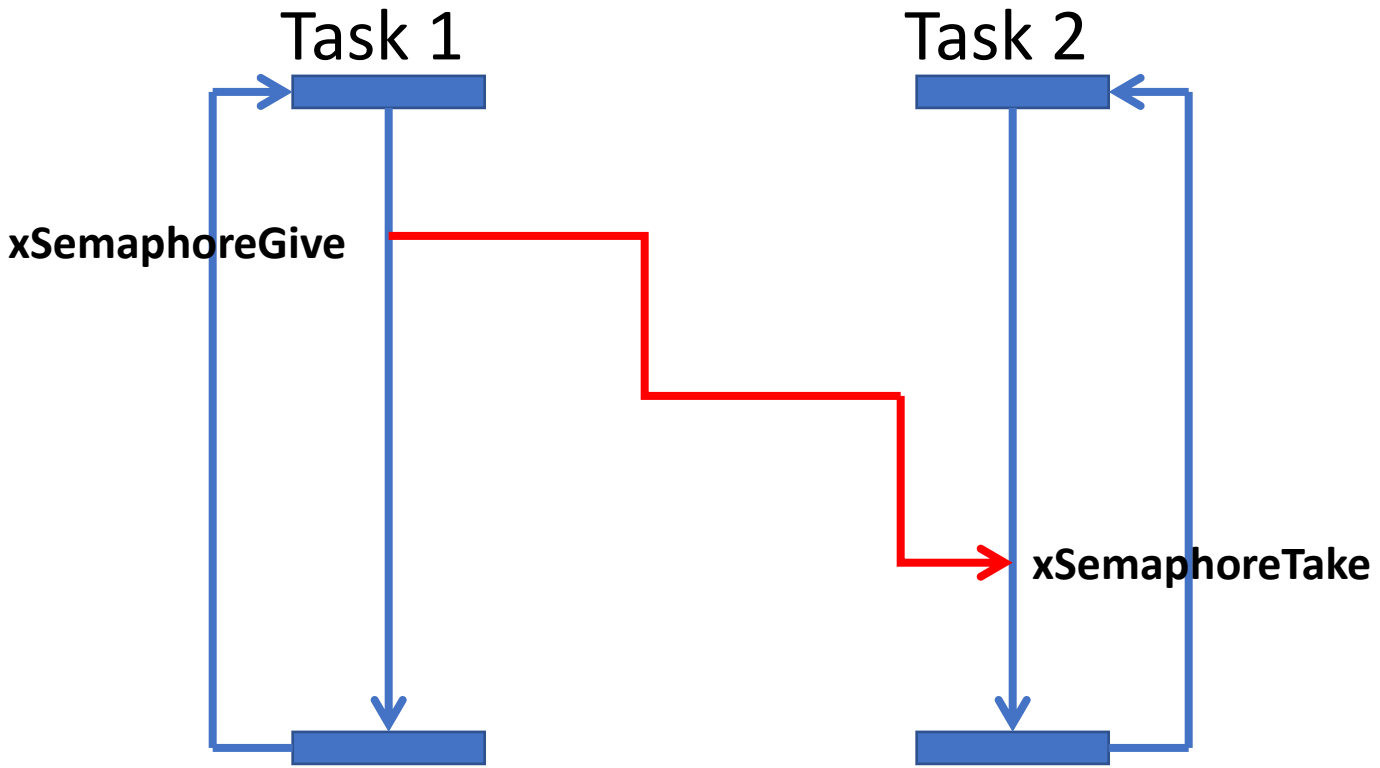
- **Tasks** creation
- Sinchronization with **SEMAPHORES**
- Message sending/receiving with **MAILBOXES**
- **Concurrent** (parallel) behavior
- Can work in a **preemptive** or **cooperative** mode.
- In a cooperative way, each task should facilitate other tasks to run by invoking **vTaskDelay(xDelay)**, **taskYIELD()** or by using **synchronization mechanisms**…

TASK CREATION: Setting up project for running parallel tasks

- Parallel tasks running **INDEPENDENTLY**

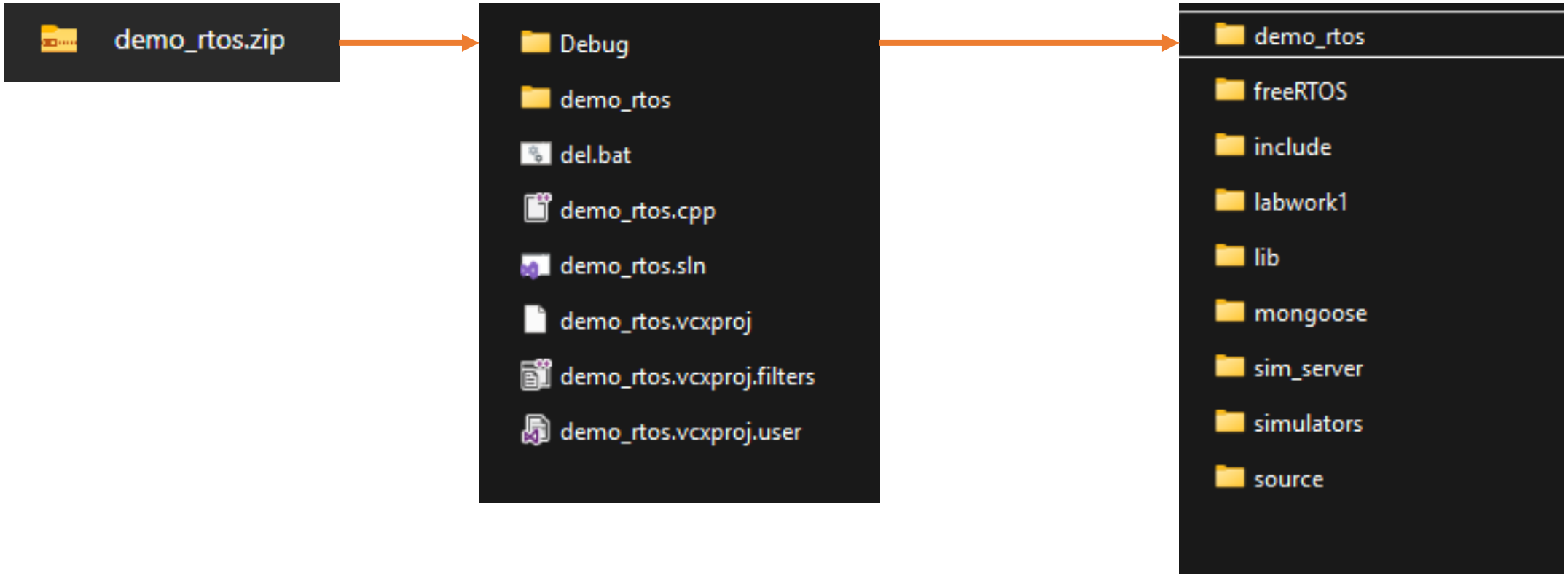


- Parallel tasks with **SYNCHRONIZATION**



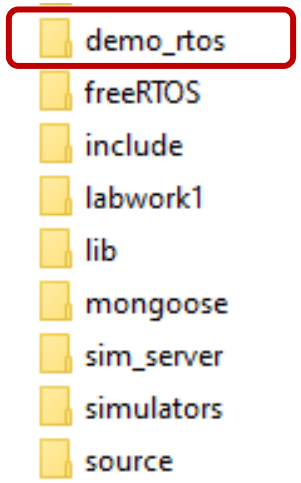
demo_rtos project

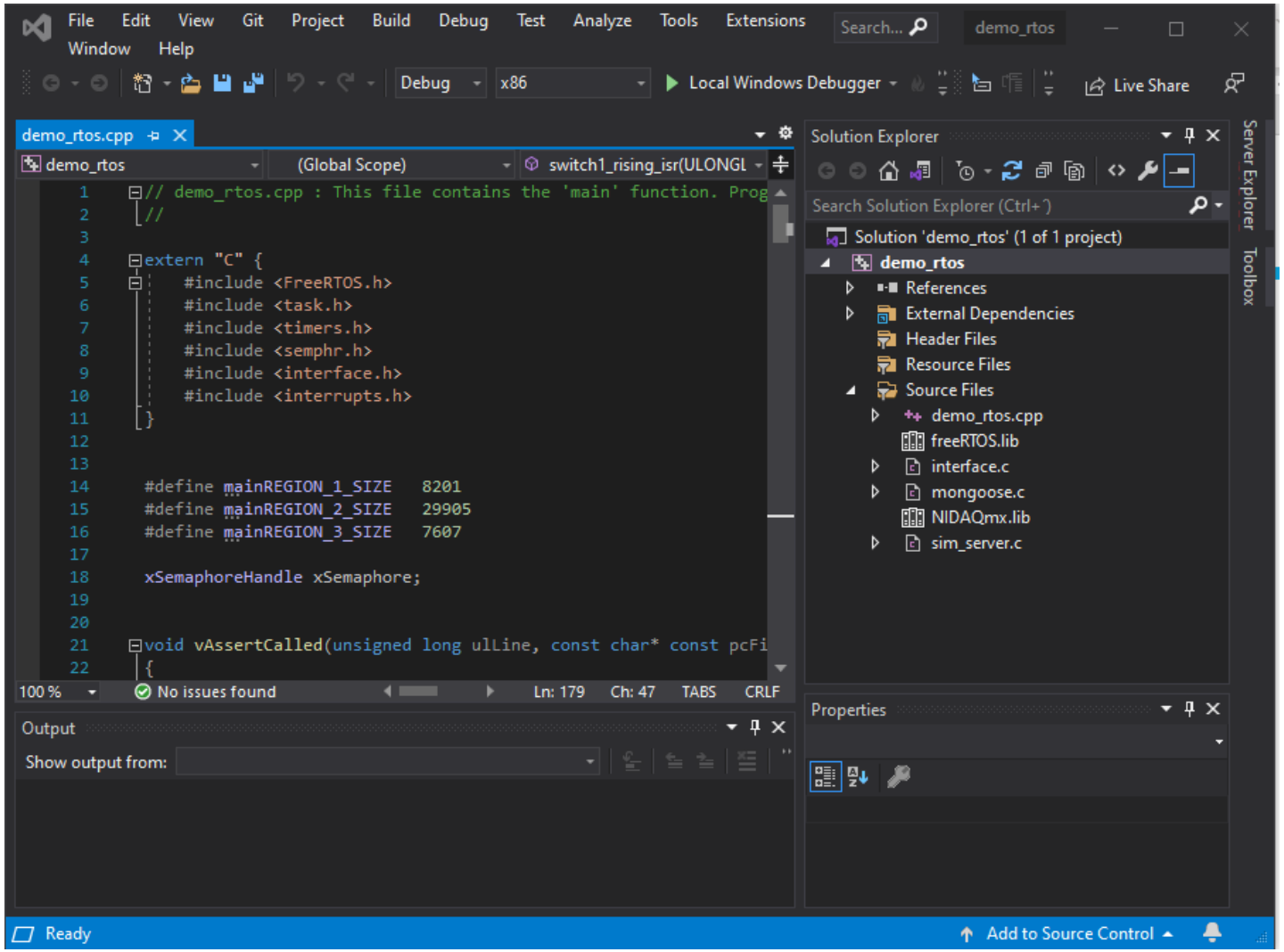
- Replace the **demo_rtos** project in your **c:\str** folder:



Open demo_rtos project

- Open the **demo_rtos** project from **c:\str** folder:





TASK CREATION: Setting up project for running parallel tasks

- Prototypes and macros for heap initialization

```
extern "C" {
   #include <FreeRTOS.h>
   #include <task.h>
   #include <timers.h>
   #include <semphr.h>
   #include <interface.h>
   #include <interrupts.h>
}

#define mainREGION_1_SIZE   8201
#define mainREGION_2_SIZE   29905
#define mainREGION_3_SIZE   7607
```

- Heap initialization

```
static void  initialiseHeap(void)
{
   static uint8_t ucHeap[configTOTAL_HEAP_SIZE];
   /* Just to prevent 'condition is always true' warnings in configASSERT(). */
   volatile uint32_t ulAdditionalOffset = 19;
   const HeapRegion_t xHeapRegions[] =
   {
        /* Start address with dummy offsetsSize */
        { ucHeap + 1,mainREGION_1_SIZE },
        { ucHeap + 15 + mainREGION_1_SIZE,mainREGION_2_SIZE },
        { ucHeap + 19 + mainREGION_1_SIZE +
                mainREGION_2_SIZE,mainREGION_3_SIZE },
        { NULL, 0 }
   };


   configASSERT((ulAdditionalOffset +
         mainREGION_1_SIZE +
         mainREGION_2_SIZE +
         mainREGION_3_SIZE) < configTOTAL_HEAP_SIZE);
        /* Prevent compiler warnings when configASSERT() is not defined. */
   (void)ulAdditionalOffset;
    vPortDefineHeapRegions(xHeapRegions);
}
```

TASK CREATION: Setting up project for running parallel tasks

- Assert function for freeRTOS error handling

```
void vAssertCalled( unsigned long ulLine, const char * const pcFileName )
{
    static BaseType_t xPrinted = pdFALSE;
    volatile uint32_t ulSetToNonZeroInDebuggerToContinue = 0;
    /* Called if an assertion passed to configASSERT() fails.  See
    http://www.freertos.org/a00110.html#configASSERT for more information. */
    /* Parameters are not used. */
    ( void ) ulLine;
    ( void ) pcFileName;
    printf( "ASSERT! Line %ld, file %s, GetLastError() %ld\r\n", ulLine, pcFileName, GetLastError());

    taskENTER_CRITICAL();
    {
       /* Cause debugger break point if being debugged. */
       __debugbreak();
       /* You can step out of this function to debug the assertion by using
          the debugger to set ulSetToNonZeroInDebuggerToContinue to a non-zero
          value. */
       while( ulSetToNonZeroInDebuggerToContinue == 0 )
       {
          __asm{ NOP };
          __asm{ NOP };
       }
    }
    taskEXIT_CRITICAL();
}
```

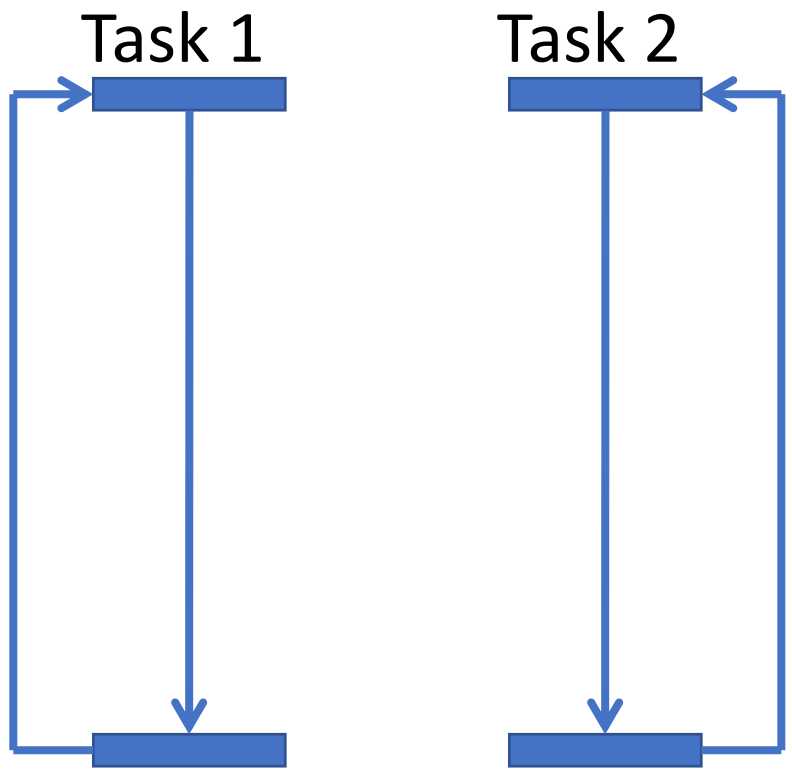
Assert in programming:

Is a piece of code that evaluates the correctness of the program at that moment.

TASK CREATION: Parallel tasks running INDEPENDENTLY

```
void vTaskCode_1( void * pvParameters )
 {
     for( ;; )     {
         printf("\nHello from TASK_1");
         // Although the kernel is in preemptive mode,
         // we should help switch to another
         // task with e.g. vTaskDelay(0) or taskYIELD()
         taskYIELD();
   }
 }
```

```
void vTaskCode_2( void * pvParameters )
{
     for( ;; )
     {
        printf("\nHello from TASK_2..");
        taskYIELD();
     }
 }
```

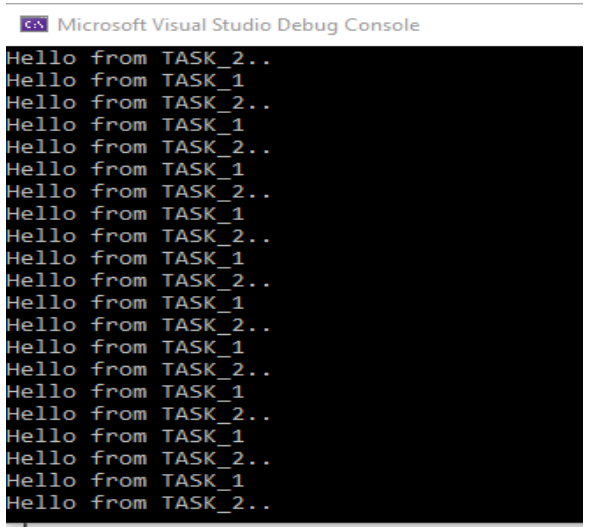


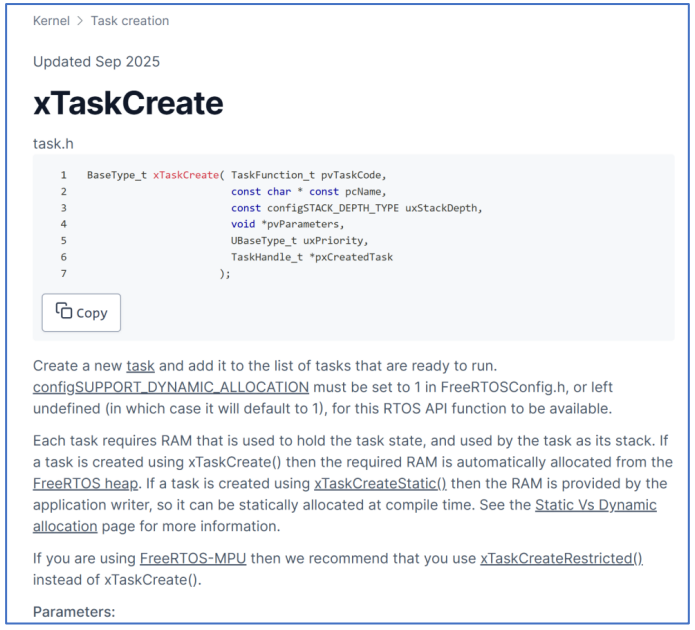
TASK CREATION: Parallel tasks running INDEPENDENTLY

```
void myDaemonTaskStartupHook(void) {
     // runs only once;
     // used for code initialization
     xTaskCreate(vTaskCode_2, "Task2", 100, NULL, 0, NULL);
     xTaskCreate(vTaskCode_1, "Task1", 100, NULL, 0, NULL);
}
```

```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```

```
xTaskCreate(
     pvTaskCode,      // pointer to function with task code (void *)
     pcName,          // task Name  (char *)
     usStackDepth,    // stack size for the task
     pvParameters,    // initial array parameters for the task
     uxPriority,      // task priority (higher value means higher priority
     pxCreatedTask    // handle to task (can be null)
)
```





Kernel > Task creation

Updated Sep 2025

xTaskCreate

task.h

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        const configSTACK_DEPTH_TYPE uxStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask
                      );
```

Copy

Create a new task and add it to the list of tasks that are ready to run. configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 in FreeRTOSConfig.h, or left undefined (in which case it will default to 1), for this RTOS API function to be available.

Each task requires RAM that is used to hold the task state, and used by the task as its stack. If a task is created using xTaskCreate() then the required RAM is automatically allocated from the FreeRTOS heap. If a task is created using xTaskCreateStatic() then the RAM is provided by the application writer, so it can be statically allocated at compile time. See the Static Vs Dynamic allocation page for more information.

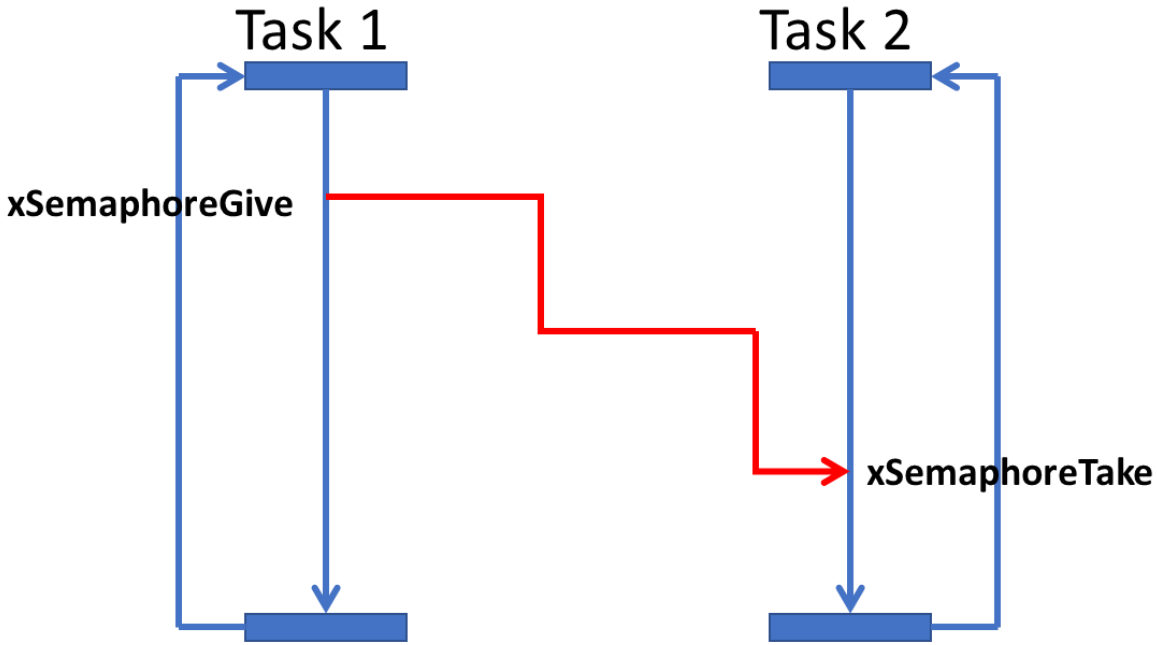
If you are using FreeRTOS-MPU then we recommend that you use xTaskCreateRestricted() instead of xTaskCreate().

Parameters:

TASK CREATION: Parallel tasks with SYNCHRONIZATION

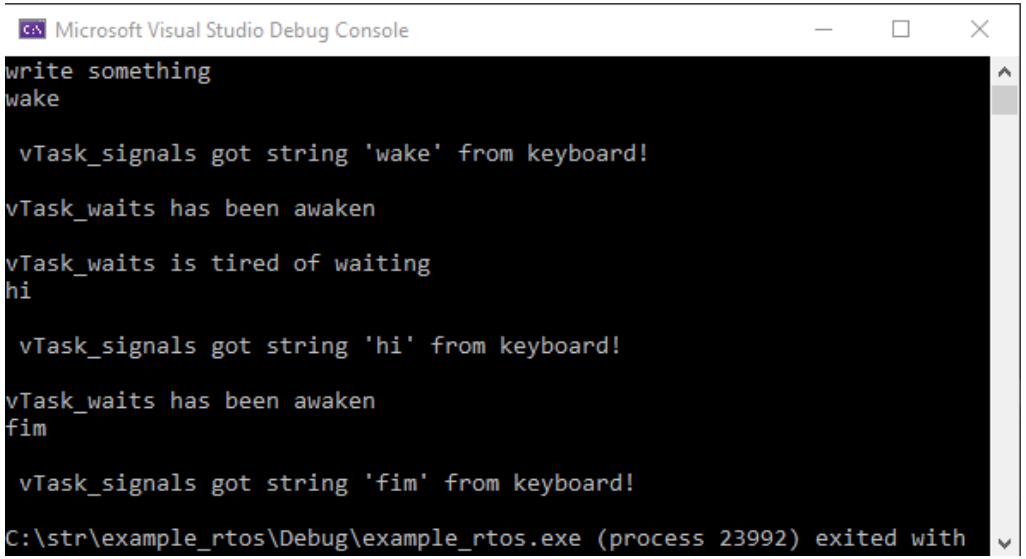
```
// declare the semaphore on top of the file, just below #defines

xSemaphoreHandle xSemaphore = NULL;

void vTask_waits(void* pvParameters)
{
    while (1) {
        if (xSemaphoreTake(xSemaphore, 10000) == pdTRUE) //P
            printf("\nvTask_waits has been awaken\n");
        else
            printf("\nvTask_waits is tired of waiting\n");
    }
}
```

```
void vTask_signals(void* pvParameters)
{
    printf("write something\n");
    while (1) {
        char ch[101];
        gets_s(ch, 100);  // input from keyboard
        printf("\n vTask_signals got string '%s' from keyboard…", ch);
        if (strcmp(ch, "fim") == 0)
            exit(0); // terminate program...
        xSemaphoreGive(xSemaphore); //V
    }
}
```



TASK CREATION: Parallel tasks with SYNCHRONIZATION

```
void myDaemonTaskStartupHook(void) {
    // runs only once;
    // used for code initialization

    /*xTaskCreate(vTaskCode_2, "Task2", 100, NULL, 0, NULL);
    xTaskCreate(vTaskCode_1, "Task1", 100, NULL, 0, NULL);*/

    xSemaphore = xSemaphoreCreateCounting(10, 0);
    xTaskCreate(vTask_waits, “vTask_waits", 100, NULL, 0, NULL);
    xTaskCreate(vTask_signals, “vTaskCode2", 100, NULL, 0, NULL);
}
```




```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```

TASK CREATION: Using freeRTOS with the Kit

```
void vTaskCylinder1(void* pvParameters)
{
    uInt8 aa;
    uInt8 sensor;
    while (TRUE)
    {
        //go front
        aa = readDigitalU8(2);
        setBitValue(&aa, 3, 0);
        setBitValue(&aa, 4, 1);
        vTaskDelay(10); // to simulate some latency
        writeDigitalU8(2, aa);
        // wait until front sensor
        sensor = readDigitalU8(0);
        while (getBitValue(sensor,3)){
            sensor = readDigitalU8(0);
            vTaskDelay(1);
        }
        // go back
        aa = readDigitalU8(2);
        setBitValue(&aa, 3, 1);
        setBitValue(&aa, 4, 0);
        writeDigitalU8(2, aa);
        vTaskDelay(10); // to simulate some latency
        // wait until back sensor
        sensor = readDigitalU8(0);
        while (getBitValue(sensor,4)){
            sensor = readDigitalU8(0);
            vTaskDelay(1);
        }
    }
}
```

```
void vTaskCylinder2(void* pvParameters)
{
    uInt8 aa;
    uInt8 sensor;
    while (TRUE)
    {
        //go front
        aa = readDigitalU8(2);
        setBitValue(&aa, 5, 0);
        setBitValue(&aa, 6, 1);
        vTaskDelay(10); // to simulate some latency
        writeDigitalU8(2, aa);
        // wait until front sensor
        sensor = readDigitalU8(0);
        while (getBitValue(sensor,1)){
            sensor = readDigitalU8(0);
            vTaskDelay(1);
        }
        // go back
        aa = readDigitalU8(2);
        setBitValue(&aa, 5, 1);
        setBitValue(&aa, 6, 0);
        writeDigitalU8(2, aa);
        vTaskDelay(10); // to simulate some latency
        // wait until back sensor
        sensor = readDigitalU8(0);
        while (getBitValue(sensor,2)){
            sensor = readDigitalU8(0);
            vTaskDelay(1);
        }
    }
}
```

TASK CREATION: Using freeRTOS with the Kit

```
void inicializarPortos() {
   printf("\nwaiting for hardware simulator...");
   printf("\nReminding: gotoCylinderStart, gotoCylinder1 and
gotoCylinder2 requires kit calibration first...");
   createDigitalInput(0);
   createDigitalInput(1);
   createDigitalOutput(2);
   writeDigitalU8(2, 0);
   printf("\ngot access to simulator...");
}
```

```
int getBitValue(uInt8 value, uInt8 bit_n)
// given a byte value, returns the value of its bit n
{
return(value & (1 << bit_n));
}
```

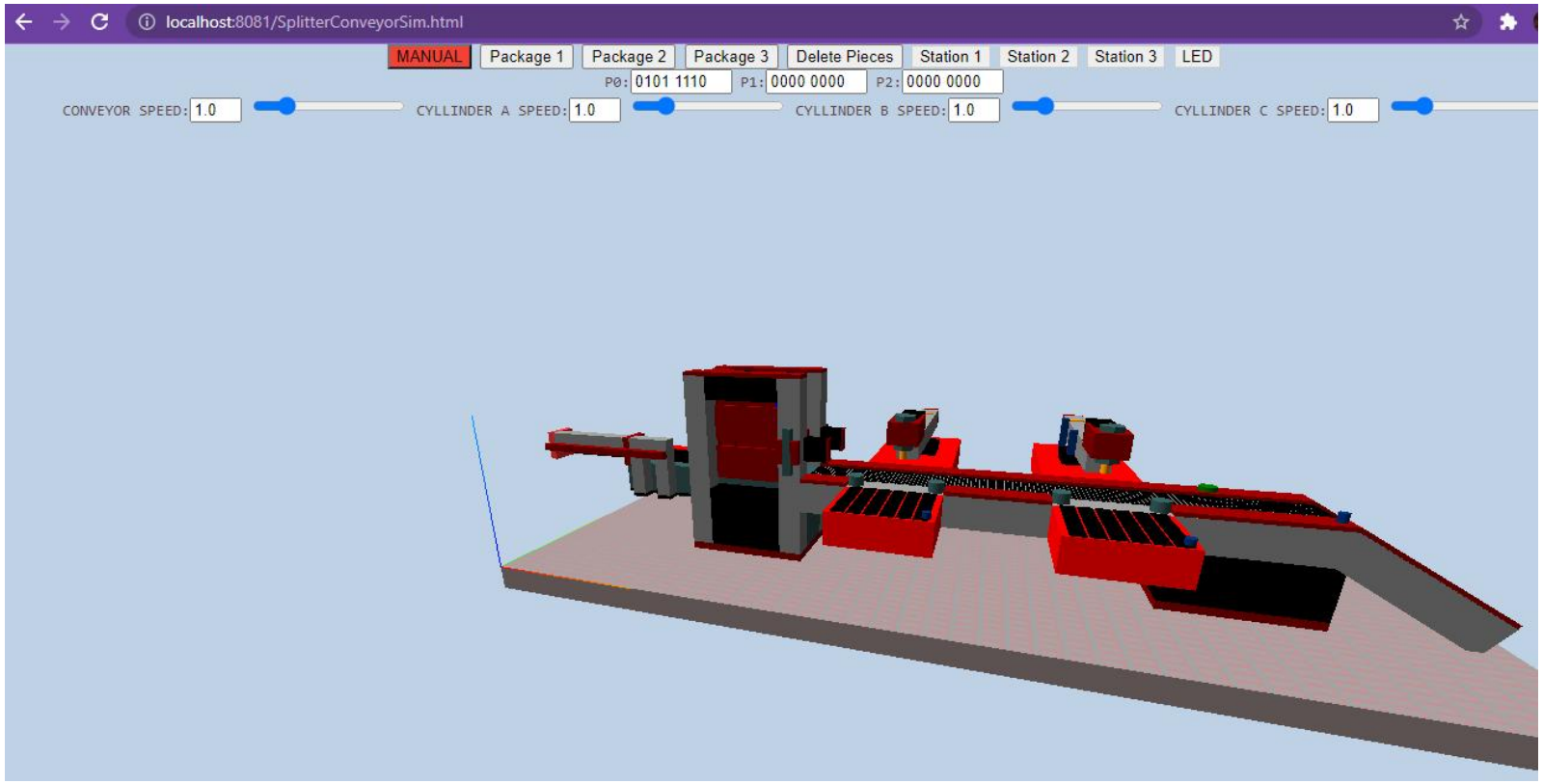
```
void setBitValue(uInt8* variable, int n_bit, int new_value_bit)
// given a byte value, set the n bit to value
{
uInt8  mask_on = (uInt8)(1 << n_bit);
uInt8  mask_off = ~mask_on;
if (new_value_bit)  *variable |= mask_on;
else                *variable &= mask_off;
}
```

```
void myDaemonTaskStartupHook(void) {
    // runs only once;
    // used for code initialization

    /*xTaskCreate(vTaskCode_2, "Task2", 100, NULL, 0, NULL);
    xTaskCreate(vTaskCode_1, "Task1", 100, NULL, 0, NULL);*/

    xSemaphore = xSemaphoreCreateCounting(10, 0);
    xTaskCreate(vTask_waits, “vTask_waits", ", 100, NULL, 0, NULL);
    xTaskCreate(vTask_signals, “vTaskCode2", 100, NULL, 0, NULL);

    inicializarPortos();

    xTaskCreate(vTaskCylinder1, "vTaskCylinder1", 100, NULL, 0, NULL);
    xTaskCreate(vTaskCylinder2, "vTaskCylinder2", 100, NULL, 0, NULL);
}
```

```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```

Using freeRTOS with the Kit

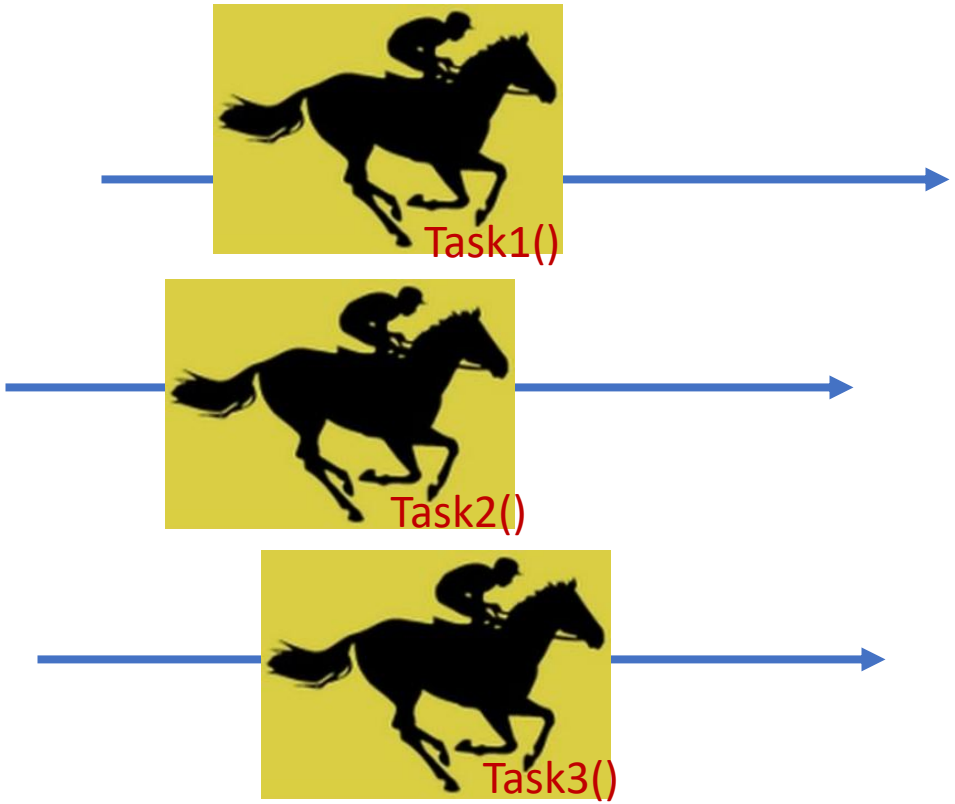
- Running the project…



Race Condition

- Several tasks running in parallel and doing simultaneous things, which may interfere with each other

Either correct or potential undesired behavior



A **race condition** or **race hazard** is the condition of an electronics, software, or other system where the system's substantive behavior is dependent on the sequence or timing of other uncontrollable events. It becomes a bug when one or more of the possible behaviors is undesirable.
The term **race condition** was already in use by 1954, for example in David A. Huffman's doctoral thesis "The synthesis of sequential switching".

https://en.wikipedia.org/wiki/Race_condition

Some issues

- Sometimes, a Cylinder stops moving.
- Erratic/irregular movements may appear!
- As Cylinder 1 and Cylinder 2 behaviors are running in “parallel”.
- Both tasks read and write values in port 2 “simultaneously”.
- And so, the values sent to port 2 are overlapping on each one!

Race Condition

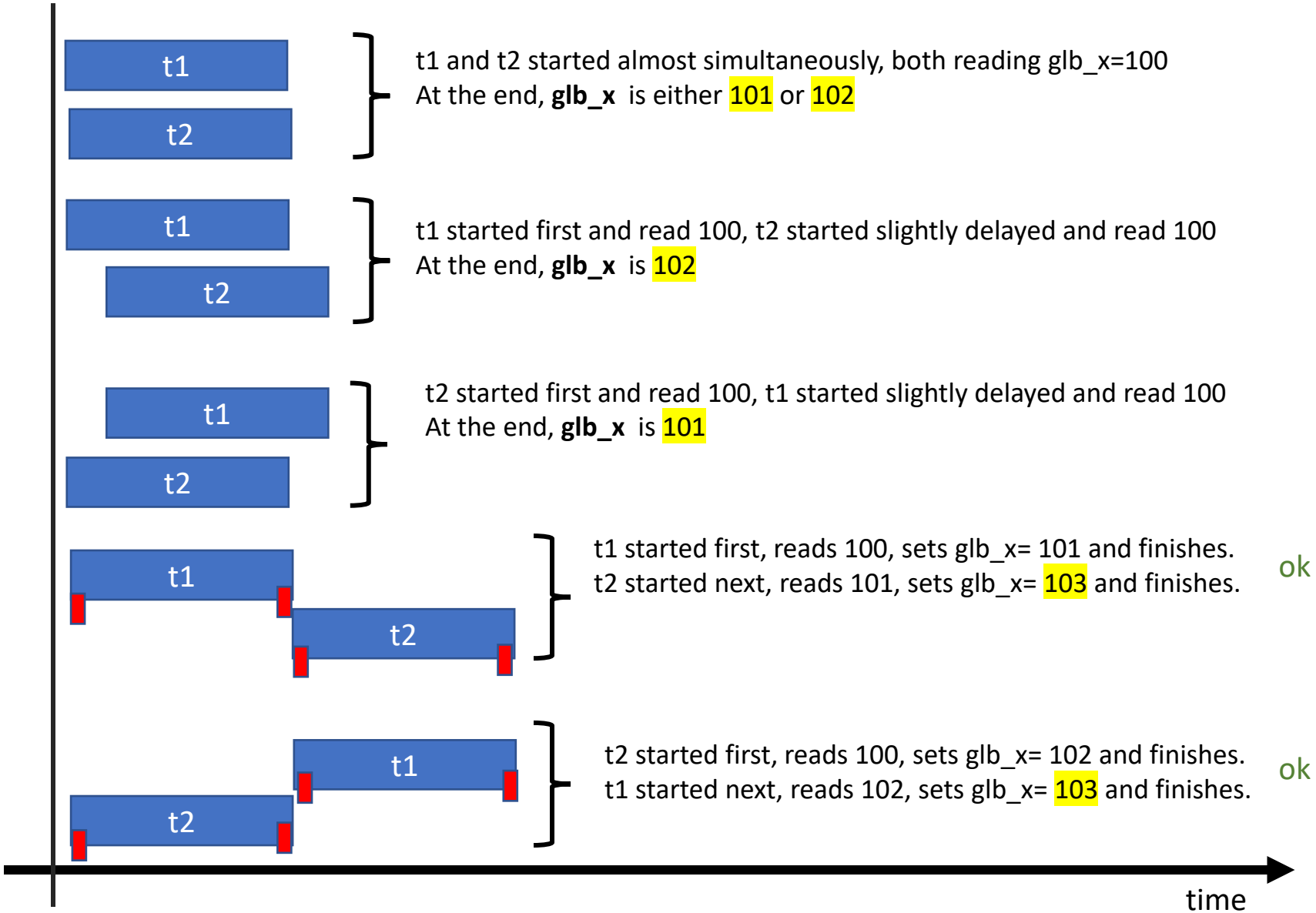
- Let's suppose glbl_x=100 and two tasks (t1 and t2) running "almost exactly" at the same time.

```
void t1() {
    int aux= glbl_x;
    aux=aux+1;
    glbl_x = aux;
}
```

```
void t2() {
    int aux= glbl_x;
    aux=aux+2;
    glbl_x = aux;
}
```

- After execution of t1 and t2, the global **glb_x** resource may assume the following values: 101, 102, or 103.

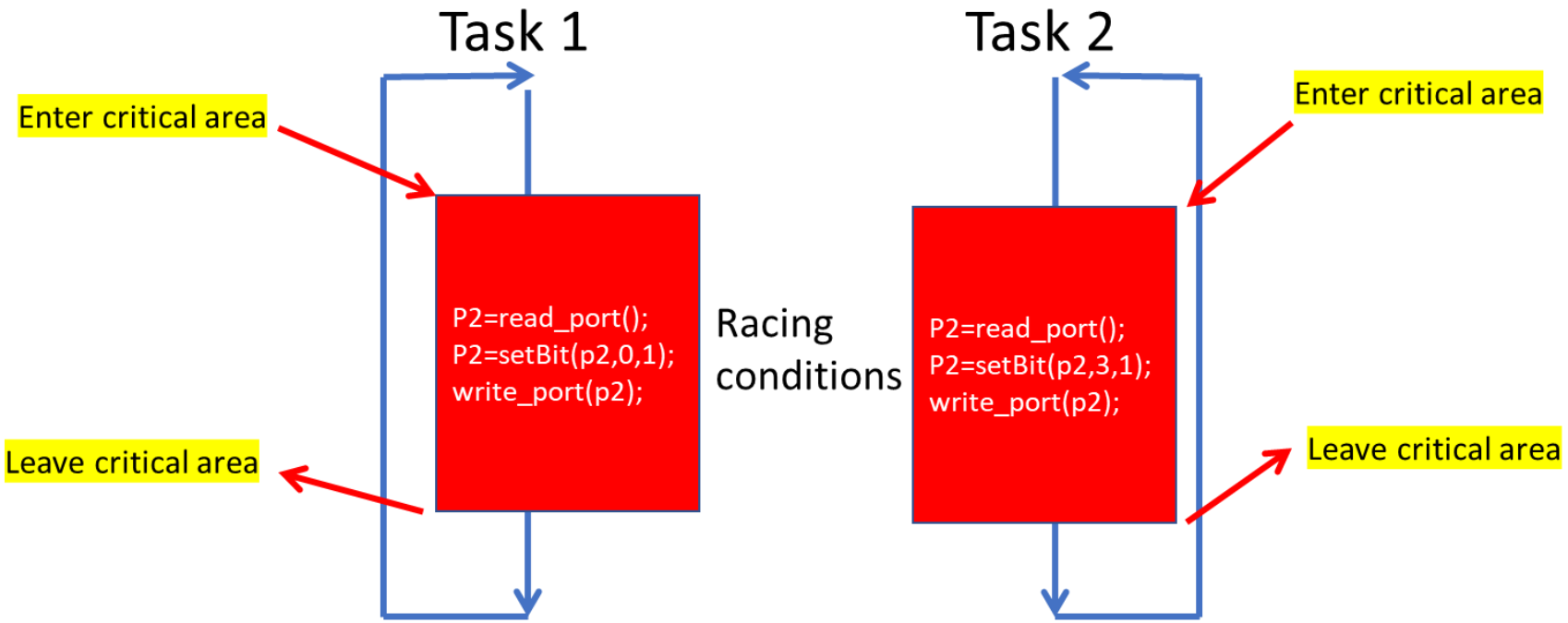
Race Condition



Synchronization to ensure that t2 starts after the t1 finishes or vice-versa

Race Condition: Splitter Conveyor (access to Port 2)

- Task 1 and Task 2 are running a fragment of code, which cannot be executed simultaneously
- We need to specify critical areas / sections.
- A critical area is a block of code which while running, everything else in the system freezes until the code inside the block finishes and execution exits from that area.



Race Condition: Splitter Conveyor (access to Port 2)

- **taskENTER_CRITICAL()** and **taskEXIT_CRITICAL()** operate at kernel level, meaning when inside a critical area, “everything” else inside the kernel stops working until exiting the critical area.
- As such, the code inside critical areas should work as fast as possible (no loops, no sleeps,… no anything causing latency).
- For less strident conditions, the recommended alternative is to model mutual exclusion with semaphores.

```
void vTaskCylinder1(void* pvParameters){
                uInt8 aa, sensor;
                 while (TRUE){
                        //go front
                        taskENTER_CRITICAL();
                         uInt8 aa = readDigitalU8(2); setBitValue(&aa, 3, 0); setBitValue(&aa, 4, 1);
                         vTaskDelay(10); // to simulate some latency
                         writeDigitalU8(2, aa);
                         taskEXIT_CRITICAL();
                          // wait until front sensor
                          sensor = readDigitalU8(0);
                          while (getBitValue(sensor,3)){
                                sensor = readDigitalU8(0); vTaskDelay(1);
                           }
                          // go back
                           taskENTER_CRITICAL();
                           aa = readDigitalU8(2); setBitValue(&aa, 3, 1);  setBitValue(&aa, 4, 0);
                            vTaskDelay(10); // to simulate some latency
                             writeDigitalU8(2, aa);
                             taskEXIT_CRITICAL();
                 // wait until back sensor
                 sensor = readDigitalU8(0);
                            while (getBitValue(sensor,4)){
                        sensor = readDigitalU8(0); vTaskDelay(1);
                  }
                }
}
```

Hook Functions

- Using **vApplicationIdleHook**
  - It runs when all tasks are occupied (waiting for something)

```
void myIdleHook(void) {
    // do something here
    printf("\nidle hook...");
    Sleep(0);
}
```

```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vApplicationIdleHook              = &myIdleHook;
    vTaskStartScheduler();
}
```

Hook Functions

- Using **vApplicationTickHook**
  - It is useful to performing periodic operations

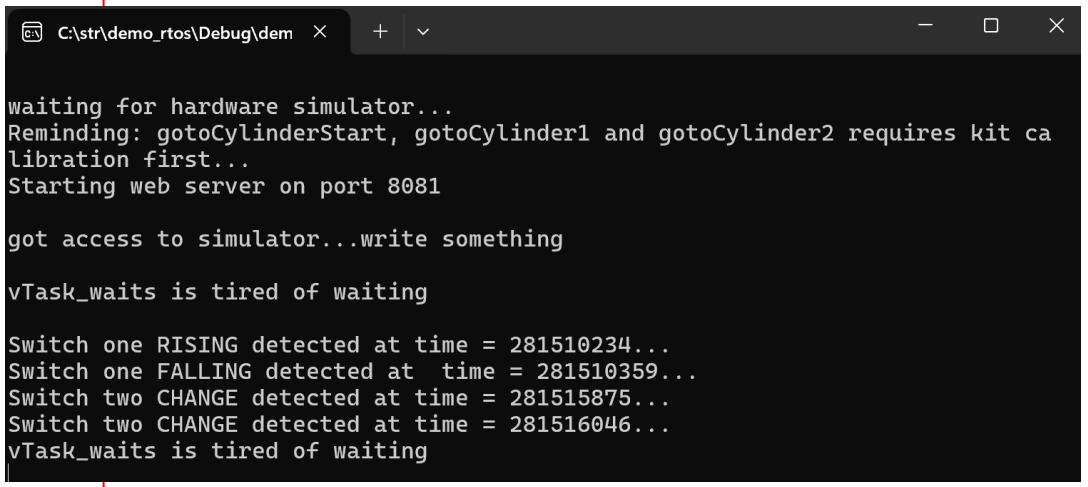
```
void myTickHook(void) {
    // do something here
    printf("\ntick hook...");
    Sleep(4000);
}
```

```
int main()
{
   initialiseHeap();
   vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
   vApplicationTickHook              = &myTickHook;
   vApplicationIdleHook              = &myIdleHook;
   vTaskStartScheduler();
}
```

Interrupts

- Using Interrupt Service Routines (ISR) (on port bit values)

```
void switch1_rising_isr(ULONGLONG lastTime) {
  // GetTickCount64() current time in miliseconds
  // since the system has started...
  ULONGLONG  time = GetTickCount64();
  printf("\nSwitch one RISING detected at time = %llu...", time);
}

void switch1_falling_isr(ULONGLONG lastTime) {
  ULONGLONG  time = GetTickCount64();
  printf("\nSwitch one FALLING detected at time = %llu...", time);
}

void switch2_change_isr(ULONGLONG lastTime) {
  ULONGLONG  time = GetTickCount64();
  printf("\nSwitch two CHANGE detected at time = %llu...", time);
}
```




```
void myDaemonTaskStartupHook(void) {
   // …code already here
   attachInterrupt(1, 4, switch1_rising_isr, RISING);
   attachInterrupt(1, 4, switch1_falling_isr, FALLING);
   attachInterrupt(1, 3, switch2_change_isr, CHANGE);
}
```

Lab Work 1

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

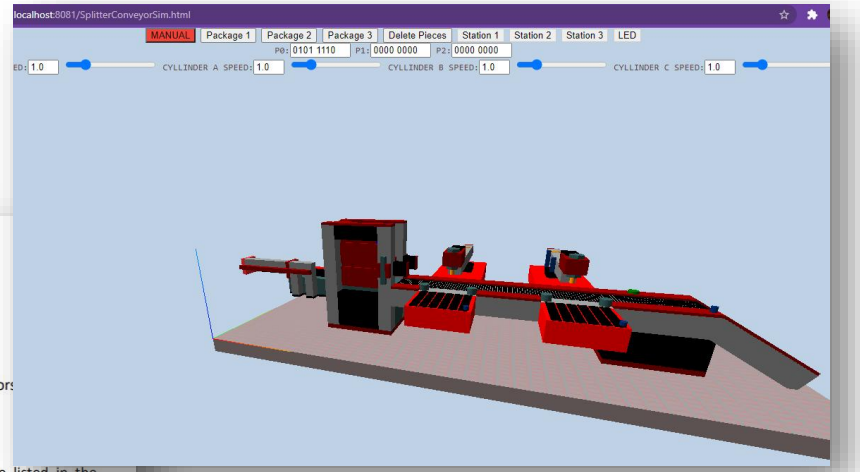
Departamento de Engenharia Electrotécnica e de Computadores

Secção de Robótica e Manufactura Integrada

Description of Lab Work nº 1	
Course	**Real-Time systems / *Sistemas de Tempo Real***
Year	2025/2026
Aim	Study concepts of Real-Time Systems and develop a project with a real-time kernel
Classes	5+1 classes x 3 hours + 15 extra hours (outside)
Delivery Date	**27/10/2023**

Concrete Objectives:

1. Interaction with physical systems (Interface boards, sensors, actuators, …)
   a. Interface board (Data acquisition)
   b. Sensors
   c. Actuators
   d. Interface functions (in C++)
2. Apply the base real-time system's concepts, namely:
   a. Task / Process: concurrency,
   b. State, event, actions (triggered by events)
   c. Synchronization and communication between tasks
   d. Accessing shared resources / exclusive access resources
3. Using Real-Time kernels
   a. freeRTOS
   b. C/C++ Language
4. Solve the problem described in Annex 1.



NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Departamento de Engenharia Electrotécnica e de Computadores

Secção de Robótica e Manufactura Integrada

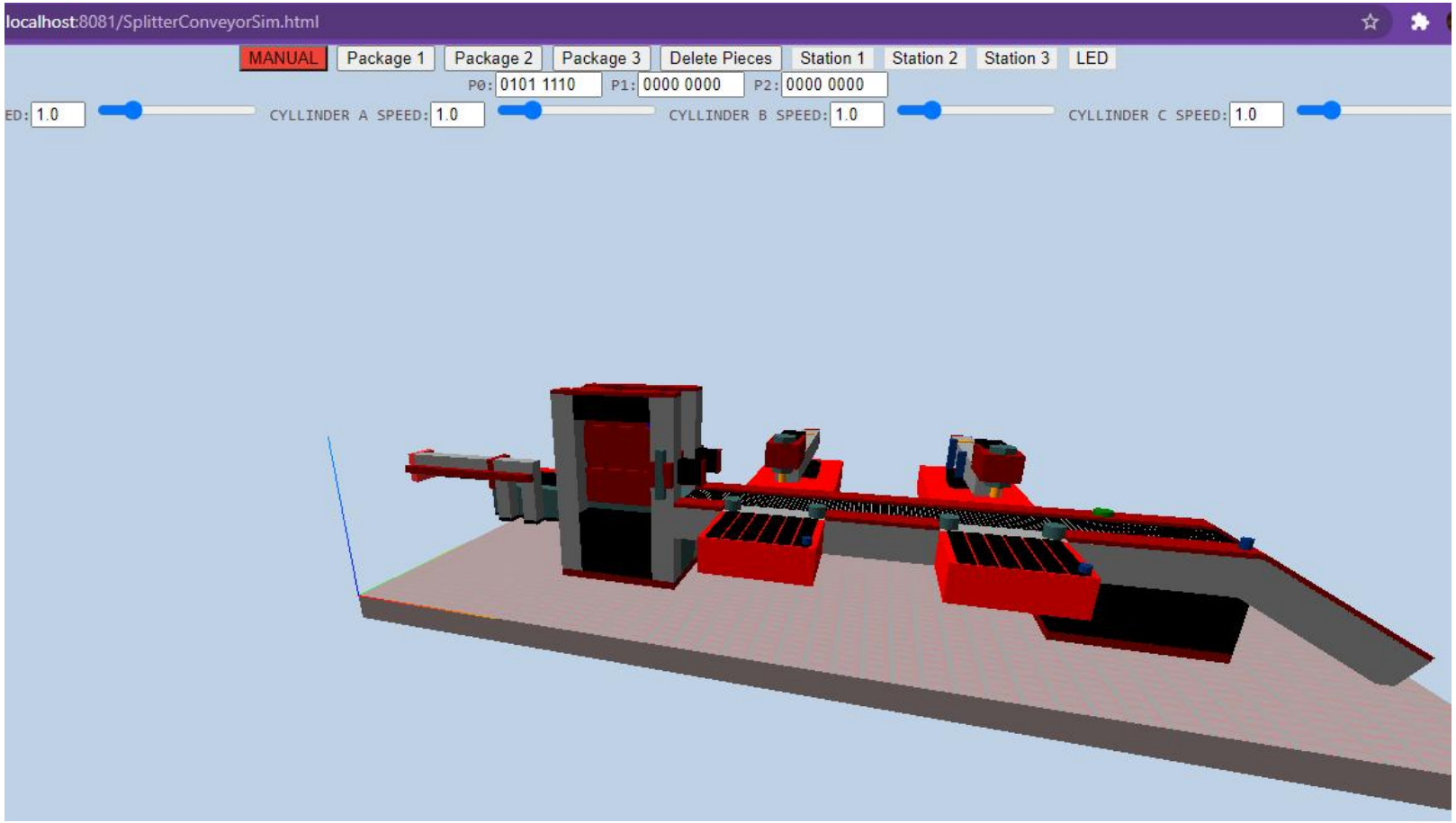
Finally, keep in mind that the system has active high and active low sensors sensors provide logical value false/0 when active, and true/1 otherwise.

The functional **requirements** to develop the **Splitter Lego Conveyor** are listed in the following table. They are split into basic or low-level requirements, for direct interaction with sensors and actuators, and high-level requirements, for storage management.

Table 1 – Needed requirements.

Req.	Description
Low-Level Requirements:	
FR1.1	Move and stop each **cylinder**
FR1.2	Move and stop moving the **conveyor**
FR1.3	Read **Brick identification** sensors
FR1.4	Detect the **button/switches** states
FR1.5	Turn **Lamp** on and off
FR1.6	Keyboard or UI interaction
High-Level requirements (operation):	
FR2	Using the keyboard, develop robust (semi-automatic) **calibration**. Using keys, the user defines the direction of the movements. During a movement, the actuator must stop when it reaches a position sensor (Cylinders)
FR3	Insert a brick into the system, which gets a brick from the entry queue and puts it in the conveyor
FR4	By pressing "s" the system Starts its working cycle. Pressing "f", the system Finishes its work.
FR5	Deliver **Brick1** for **Dock 1**, **Brick2** for **Dock 2**, and **Brick3** to **DockEnd**
FR6	Detection of **buttons** P1.4 and P1.3 pressed by human operator, at dock 1 and dock 2 to force specific bricks to the corresponding docks
FR7	Flash lamp at P2.7 once when delivering sequence at Dock 1 or Dock 2

Lab Work 1



- Start thinking of which tasks are needed to be executed synchronously or in parallel.
- Tip: start by implementing **task_gotoCylinderStart** and **task_gotoCylinder1**…
- Draw a first diagram illustrating the tasks and the interactions between them.

AWESOME! LET'S USE TASKS AND THEIR MECHANISMS TO PLAY WITH YOUR LAB 1 SYSTEM IN

REAL-TIME SYSTEMS!

RELAX, EXPLORE, HAVE FUN, AND ENJOY THE RIDE!

KEEP UP THE GOOD WORK!